

CENTRO UNIVERSITÁRIO MAURÍCIO DE NASSAU

DESENVOLVIMENTO DO COMPILADOR LUDICODE

Uma Linguagem de Programação Estruturada em Português

Equipe de Desenvolvimento:

Abraão Araujo dos Santos
Erick Alves de Souza
Evandro França da Silva Filho
Gabriel Marques Barbosa de Santana
Gabriella Maria Nascimento da Silva
Jonas Kauan Pereira Novaes
Mateus De Miranda Santos Moura
Matheus Pinheiro da Cunha

Sumário

1	Introdução	2
2	Estrutura e Recursos da Linguagem	2
3	Gramática Formal da Linguagem	3
4	Desenvolvimento do Sistema	4
4.1	Pipeline de Processamento	4
4.2	Análise Léxica (<code>lexer.py</code>)	4
4.3	Análise Sintática (<code>parse.py</code>) e Árvore Abstrata (AST)	4
4.4	Interpretador Executável (<code>interpreter.py</code>)	5
5	Tratamento de Erros e Análise Semântica	6
6	Testes Executados e Saídas do Sistema	6
6.1	Teste do Laço <code>enquanto</code>	6
6.2	Teste Condicional Estruturado	7
6.3	Exemplos de Captura de Erros Semânticos	8
7	Conclusão	8

1 Introdução

Para que o computador compreenda as instruções, o projeto utiliza uma arquitetura clássica de compiladores, desenvolvida em Python com o auxílio da biblioteca SLY. O processo funciona como uma linha de montagem: primeiro, o sistema lê o texto e o organiza em pedaços lógicos (análise léxica); em seguida, ele verifica se a estrutura dessas frases obedece às regras da linguagem para montar um mapa de execução, chamado de Árvore Sintática Abstrata (análise sintática); por fim, ele valida se o comando faz sentido matematicamente e logicamente (validação semântica) antes de efetivamente executá-lo (interpretador).

2 Estrutura e Recursos da Linguagem

A linguagem LudiCode possui uma sintaxe imperativa e fortemente estruturada por blocos delimitados por chaves (`{}`). Os principais recursos oferecidos e suportados pela arquitetura estável do interpretador englobam:

- **Declaração e Atribuição Dinâmica:** Uso da palavra reservada `var` para introdução de novos identificadores na tabela de símbolos, permitindo reatribuições subsequentes.
- **Controle de Fluxo Condicional:** Estruturas compostas do tipo `se` e `senao` avaliadas a partir de expressões relacionais booleanas.
- **Laços de Repetição Iterativos:** Suporte para laços condicionais pré-testados (`enquanto`) e blocos de repetição de iteração direta (`repita`).
- **Mecanismo de Saída Embutido:** Função nativa de terminal `mostrar()` para fins de depuração e visualização de dados textuais e numéricos.

O exemplo de listagem a seguir ilustra o comportamento típico de validação condicional em código-fonte LudiCode:

```
1 var idade = 18
2
3 se (idade >= 18) {
4     mostrar("Maior de idade")
5 } senao {
6     mostrar("Menor de idade")
7 }
```

Listing 1: Exemplo de estrutura de controle condicional

3 Gramática Formal da Linguagem

A sintaxe da linguagem LudiCode é especificada formalmente por meio da Notação Backus-Naur (**BNF**), descrevendo com precisão as produções gramaticais aceitas e validadas pelo analisador sintático (`parse.py`).

```
<program>      ::= <statement>
                  | <program> <statement>

<statement>    ::= "var" <id> "=" <expr>
                  | <id> "=" <expr>
                  | "mostrar" "(" <expr> ")"
                  | "se" "(" <expr> ")" <bloco>
                  | "se" "(" <expr> ")" <bloco> "senao" <bloco>
                  | "enquanto" "(" <expr> ")" <bloco>
                  | "repita" <numero> <bloco>

<bloco>        ::= "{" <program> "}"

<expr>         ::= <expr> "+" <expr>
                  | <expr> "-" <expr>
                  | <expr> "*" <expr>
                  | <expr> "/" <expr>
                  | <expr> "==" <expr>
                  | <expr> "<" <expr>
                  | <expr> ">" <expr>
                  | <expr> "<=" <expr>
                  | <expr> ">=" <expr>
                  | <numero>
                  | <texto>
                  | "verdadeiro"
                  | "falso"
                  | <id>

<numero>       ::= [0-9]+ | [0-9]+ "." [0-9]+
<texto>        ::= "'" <qualquer-char>* "'"
<id>           ::= [a-zA-Z_][a-zA-Z0-9_]*
```

Listing 2: Especificação BNF da Linguagem LudiCode

4 Desenvolvimento do Sistema

A implementação foi dividida modularmente em quatro arquivos funcionais independentes, cujas atribuições respeitam o pipeline tradicional de compilação.

4.1 Pipeline de Processamento

O fluxo de processamento de dados ocorre de maneira síncrona e hierárquica, transformando a sequência de caracteres textuais brutos em uma execução semântica controlada, conforme ilustrado no diagrama a seguir:



4.2 Análise Léxica (lexer.py)

O analisador léxico (`LudiCodeLexer`) converte o fluxo de caracteres de entrada em um conjunto ordenado de *tokens*. Palavras reservadas e identificadores genéricos são processados por meio de expressões regulares combinadas com um dicionário interno de mapeamento estrito.

A tabela a seguir consolida as palavras-chave de controle implementadas de forma estável no arquivo `lexer.py`:

Token Léxico	Palavra Reservada	Descrição Funcional
VAR	var	Instanciação de identificadores na memória.
SE	se	Desvio condicional simples.
SENAO	senão / senao	Bloco alternativo de desvio condicional.
ENQUANTO	enquanto	Laço de repetição sob pré-condição.
REPITA	repita	Laço de iteração quantificável.
MOSTRAR	mostrar	Função nativa de impressão em console.
VERDADEIRO	verdadeiro	Literal lógico booleano positivo.
FALSO	falso	Literal lógico booleano negativo.

Identificadores textuais genéricos (ID) que coincidem com nomes comuns de variáveis são capturados pela regra base `r'[a-zA-Z_][a-zA-Z0-9_]*'`, sendo confrontados com o dicionário de palavras reservadas antes da classificação definitiva.

4.3 Análise Sintática (parse.py) e Árvore Abstrata (AST)

A análise sintática baseia-se em regras de produção LALR(1) parametrizadas por decoradores de escopo fornecidos pela biblioteca `SLY`. A resolução de ambiguidades gramaticais

de expressões (como precedência aritmética e associatividade esquerda) é explicitamente estabelecida pela tupla de precedência:

```

1 precedence = (
2     ('nonassoc', 'IGUAL', 'MENOR', 'MAIOR', 'MENOR_IGUAL', '
3     MAIOR_IGUAL'),
4     ('left', '+', '-'),
5     ('left', '*', '/'),
6 )

```

Listing 3: Declaração de precedência estática no parser

Ao processar com sucesso a entrada, o parser sintetiza uma Árvore Sintática Abstrata (**AST**) estruturada unicamente em **tuplas aninhadas** nativas do Python. Essa padronização permite que a estrutura interna seja compacta e de fácil varredura pelo interpretador.

Abaixo estão listados os formatos estruturais das tuplas geradas na AST para as principais construções sintáticas:

Construção Sintática	Estrutura do Nó na AST (Tupla)
Declaração de variável	('declaracao_var', nome, expressao)
Atribuição simples	('atribuicao', nome, expressao)
Comando Mostrar	('mostrar', expressao)
Condicional Simples	('se', condicao, bloco_verdadeiro)
Condicional Composta	('se', condicao, bloco_verd, bloco_falso)
Laço Enquanto	('enquanto', condicao, bloco_corpo)
Laço Repita	('repita', numero, bloco_corpo)
Operação Binária	('operacao', operador, esq, dir)

4.4 Interpretador Executável (interpreter.py)

O interpretador (`LudiCodeInterpreter`) varre recursivamente a AST com suporte a tabelas de símbolos dinâmicas (`SymbolTable`) implementadas em dicionários Python. As variáveis locais sofrem checagem semântica em tempo de execução, garantindo isolamento de estado e controle de tipos primitivos (tipagem dinâmica baseada nos tipos nativos `int`, `float`, `str` e `bool`).

O interpretador gerencia também os laços por meio de um teto de segurança contra loops infinitos (`loop_limit = 10000`), prevenindo estouros de pilha ou congelamento do terminal de simulação educacional.

5 Tratamento de Erros e Análise Semântica

O sistema valida a semântica do programa de forma ativa durante a interpretação dos nós da AST. Caso alguma regra seja violada, o interpretador interrompe a execução e lança uma exceção customizada do tipo `SemanticError`, detalhando o erro em linguagem clara para o aluno.

As seguintes violações semânticas são interceptadas e tratadas nativamente pelo código atual:

1. **Erro de Dupla Declaração:** Lançado quando o comando `var` tenta reinicializar um identificador já existente na `SymbolTable`.
2. **Variável Não Declarada:** Ocorre ao tentar ler ou atribuir um valor a uma variável que não foi previamente instanciada com `var`.
3. **Divisão por Zero:** Interceptado antes da execução da operação aritmética básica de divisão (`/`), evitando a quebra abrupta do interpretador em Python.
4. **Incompatibilidade de Tipos Primitivos:** Ocorre ao tentar realizar operações matemáticas (como a subtração `-`, multiplicação `*` ou divisão `/`) envolvendo strings e números inteiros/decimais simultaneamente.

6 Testes Executados e Saídas do Sistema

Os scripts de teste homologados reproduzem os logs reais do terminal obtidos durante as sessões de validação do interpretador através do módulo `main.py`.

6.1 Teste do Laço enquanto

O script `teste_enquanto.ludi` avalia uma variável contadora incremental executada três vezes consecutivas.

```
1 var x = 0
2
3 enquanto (x < 3) {
4     mostrar(x)
5     x = x + 1
6 }
```

Listing 4: tests/teste_enquanto.ludi

Log de saída e depuração gerado no terminal:

```

--- [1] Iniciando Analise Lexica ---

--- [2] Iniciando Analise Sintatica ---
Arvore de Sintaxe Abstrata (AST) gerada:
[('declaracao_var', 'x', ('numero', 0)),
 ('enquanto',
  ('operacao', '<', ('variavel', 'x'), ('numero', 3)),
  [('mostrar', ('variavel', 'x')),
   ('atribuicao', 'x', ('operacao', '+', ('variavel', 'x'), ('
      numero', 1)))]
)]

--- [3] Execucao / Interpretador ---

Saida do programa:
0
1
2

Estado final:
Variaveis: {'x': 3}
Sensores: {}
Motores: {}
Eventos: []

```

Listing 5: Saída do console para o laço enquanto

6.2 Teste Condicional Estruturado

O arquivo `teste_se.ludi` valida as ramificações relacionais maior ou igual.

```

1 var idade = 18
2
3 se (idade >= 18) {
4     mostrar("Maior de idade")
5 } senao {
6     mostrar("Menor de idade")
7 }

```

Listing 6: tests/teste_se.ludi

Log de saída gerado no terminal:

```
Saida do programa:
Maior de idade

Estado final:
Variaveis: {'idade': 18}
Sensores: {}
Motores: {}
Eventos: []
```

Listing 7: Saída do console para desvio condicional

6.3 Exemplos de Captura de Erros Semânticos

Abaixo constam as respostas nativas emitidas pelo terminal em cenários de códigos-fonte intencionalmente malformados semanticamente:

```
1 var x = "abc" - 1
```

Listing 8: tests/teste_tipos.ludi

```
Erro durante a execucao: Tipos incompativeis para o operador '-':
str e int.
```

Listing 9: Log de erro para tipos incompatíveis

```
1 var x = 10 / 0
```

Listing 10: tests/teste_divisao_zero.ludi

```
Erro durante a execucao: Divisao por zero.
```

Listing 11: Log de erro para divisão por zero

7 Conclusão

O desenvolvimento do LudiCode permitiu aplicar na prática conceitos estudados na disciplina de Compiladores. O projeto mostra como uma linguagem de programação pode ser criada a partir da definição de tokens, regras gramaticais e mecanismos de execução. Durante o trabalho, foram implementadas etapas fundamentais, como análise léxica, análise sintática, construção da AST e interpretação dos comandos. Cada uma dessas etapas possui uma função específica dentro do funcionamento do sistema. Além do aspecto técnico, o projeto também possui uma proposta educacional, pois utiliza comandos em

português para facilitar o aprendizado de programação. Dessa forma, o LudiCode demonstra como conceitos de compiladores podem ser aplicados em uma ferramenta voltada ao ensino de lógica computacional e robótica.